TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN

KHOA CÔNG NGHỆ THÔNG TIN

--------- oOo ---------



BÀI TẬP THỰC HÀNH

TRUYỀN THÔNG TRONG IoT

TRÌNH ĐỘ: ĐẠI HỌC

NGÀNH ĐÀO TẠO: CÔNG NGHỆ THÔNG TIN

Hưng Yên – Tháng 01 năm 2025

BÀI THỰC HÀNH SỐ 13: GIAO THỨC TẦNG ỨNG DỤNG - CoAP

A. MỤC TIÊU BÀI THỰC HÀNH

Sau bài thực hành này sinh viên có thể:

- Trình bày được vai trò và vị trí của giao thức CoAP trong ngăn xếp giao thức IoT và sự khác biệt cơ bản giữa CoAP với HTTP và MQTT.
- Hiểu và giải thích được bản chất cơ chế truyền tin của CoAP trên nền UDP, bao gồm các loại message (CON, NON, ACK) và cơ chế đảm bảo độ tin cậy ở mức bản tin.
- Phân tích được cấu trúc gói tin CoAP (Header, Message ID, Token, Options, Payload) thông qua việc bắt và phân tích gói tin bằng Wireshark.
- So sánh và đánh giá được hai mô hình tương tác Pull (GET định kỳ) và Push (Observe) trong các hệ thống IoT, từ đó lựa chọn cơ chế phù hợp cho từng bài toán thực tế.
- Hiểu và giải thích được cơ chế Blockwise Transfer trong CoAP và lý do giao thức này không sử dụng IP fragmentation khi truyền dữ liệu lớn.
- Vận dụng được kiến thức đã học để thiết kế và triển khai hệ thống CoAP Client – Server đơn giản, có khả năng mở rộng cho các ứng dụng IoT thực tế như giám sát cảm biến, cập nhật firmware hoặc tích hợp với hệ thống Web.

B. ĐIỀU KIỆN THỰC HÀNH

Để thực hiện vài thực hành này, sinh viên cần:

1. Kiến thức yêu cầu
    - Kiến thức cơ bản về mạng máy tính, bao gồm:
        - Mô hình TCP/IP
        - Nguyên lý hoạt động của giao thức TCP và UDP

- Kiến thức về giao thức tầng ứng dụng:
    - Hiểu khái niệm Client – Server
    - Hiểu các giao thức tầng ứng dụng HTTP/HTTPS, MQTT,..

2. Phần cứng
    - 01 bo mạch vi điều khiển thuộc họ ESP: ESP32 / ESP8266
    - 01 máy tính cá nhân (PC/Laptop)
3. Phần mềm
    - Arduino IDE hoặc Platform IO
    - Visual Studio Code cài đặt môi trường NodeJS
    - Phần mềm Wireshark

C. TÀI NGUYÊN THAM CHIẾU

Để hoàn thành tốt bài thực hành này sinh viên nên tham khảo các tài nguyên sau:

STT	Tên tài nguyên	Mô tả tài nguyên
1	Bai tap thuc hanh 13.pdf	Tài liệu bài thực hành số 13
2	Các bài giảng về CoAP	Slide bài giảng về CoAP

D. YÊU CẦU BÀI THỰC HÀNH

Bài 13.1. Phân tích cơ chế Message và độ tin cậy (Reliability) trên nền UDP

E. HƯỚNG DẪN THỰC HIỆN

Bài 13.1. Phân tích cơ chế Message và độ tin cậy (Reliability) trên nền UDP

Mục tiêu

Trong bài thực hành này, sinh viên sẽ xây dựng một hệ thống CoAP Client – Server đơn giản để gửi dữ liệu cảm biến nhiệt độ. Sinh viên sẽ lần lượt sử dụng hai loại bản tin NON (Non-confirmable) và CON (Confirmable) để so sánh cơ chế truyền tin. Thông qua việc cố

tình tạo tình huống mất ACK, sinh viên sẽ sử dụng Wireshark để quan sát, phân tích và hiểu rõ cơ chế retransmission, Message ID và ACK của CoAP.

Chuẩn bị

- *Phần cứng*
  - Module ESP32/ESP8266
  - Cáp USB
- *Phần mềm*
  - Arduino IDE hoặc Platform IO
  - Thư viên **coap-simple-library** (Arduino CoAP)
  - Phần mềm Wireshark

Lưu ý: ESP, CoAP Server và máy chạy Wireshark phải cùng mạng WiFi LAN

Gợi ý thực hiện

Bước 1 – Xây dựng CoAP Server (Xây dựng trên Node.js)

```
const coap = require('coap');
const server = coap.createServer();
server.on('request', (req, res) => {
  console.log('Received:', req.payload.toString());

  // CỐ TÌNH KHÔNG PHẢN HỒI ACK TRONG LẦN ĐẦU
  // res.end('OK');  // Comment dòng này trong lần test đầu
});
server.listen(() => {
  console.log('CoAP server started');
```

```
});
```

Bước 2 – Viết CoAP Client trên ESP8266/ESP32

- Code gửi NON message

```
#if defined(ESP8266)
  #include <ESP8266WiFi.h>
#else
  #include <WiFi.h>
#endif

#include <WiFiUdp.h>
#include <coap-simple.h>

/* ========== CONFIG ========== */
const char* WIFI_SSID = "YOUR_WIFI_SSID";
const char* WIFI_PASS = "YOUR_WIFI_PASSWORD";

/* CoAP Server (Receiver) */
IPAddress coap_server_ip(192, 168, 1, 100);
const int coap_port = 5683;

/* CoAP Resource */
const char* coap_resource = "sensor/temp";
```

```
/* Send interval (ms) */
const unsigned long SEND_INTERVAL = 5000;
/* ============================== */

WiFiUDP udp;
Coap coap(udp);

unsigned long lastSend = 0;

/* ============================== */
void setup_wifi() {
  Serial.print("Connecting WiFi");
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());
}
```

```
/* ================================ */

void setup() {
  Serial.begin(115200);
  setup_wifi();

  coap.start();
  Serial.println("CoAP client started");
}

/* ================================ */

void loop() {
  if (millis() - lastSend >= SEND_INTERVAL) {
    lastSend = millis();

    /* Payload mẫu */
    String payload = "temp=30.5";

    Serial.print("Sending CoAP NON POST to ");
    Serial.print(coap_resource);
```

```
    Serial.print(" payload: ");
    Serial.println(payload);


    coap.send(
      coap_server_ip,
      coap_port,
      coap_resource,
      COAP_NON,          // Non-confirmable
      COAP_POST,         // Method
      (uint8_t*)payload.c_str(),
      payload.length()
    );
  }
}
```

- Code gửi CON message (thay đổi message type)

```
coap.send(
  coap_server_ip,
  coap_port,
  coap_resource,
  COAP_CON,                // Confirmable message
  COAP_POST,               // Method: POST
  (uint8_t*)payload.c_str(), // Payload
```

```
  payload.length()          // Độ dài payload
);
```

Bước 3 – Bắt gói tin bằng Wireshark

- Lọc: **coap**

Bước 4 – Tạo tình huống lỗi

1. Server không phản hồi
2. Quan sát:
    - NON: có gửi lại không?
    - CON: gửi lại bao nhiêu lần?
3. So sánh MID giữa các gói

Câu hỏi kiểm tra

- Khi gửi NON message, nếu server không phản hồi, điều gì xảy ra? Wireshark thể hiện điều này như thế nào?
- Khi gửi CON message mà không nhận ACK:
    - Có retransmission không?
    - Message ID có thay đổi không?
    - Sau bao lâu thì gửi lại?
- So sánh cơ chế đảm bảo tin cậy của CoAP (CON/ACK) với TCP (ACK/SEQ). CoAP không đảm bảo điều gì mà TCP có?
- Trong hệ thống IoT:
    - Trường hợp nào nên dùng NON? Lấy ví dụ?
    - Trường hợp nào bắt buộc dùng CON? Lấy ví dụ?

Sinh viên hoàn thành theo hướng dẫn và trả lời các câu hỏi trên.

Bài 13.2. So sánh Pull (GET định kỳ) và Push (Observe) trong CoAP

Mục tiêu

Trong bài thực hành này, sinh viên sẽ xây dựng một hệ thống CoAP gồm CoAP Server chạy trên Node.js và CoAP Client chạy trên ESP8266/ESP32. Sinh viên sẽ triển khai hai cơ chế giám sát tài nguyên cảm biến nhiệt độ:

- GET định kỳ (Polling / Pull)
- Observe (Push)

Thông qua việc bắt gói tin bằng Wireshark, sinh viên sẽ phân tích sự khác biệt về số lượng gói tin, Message ID, Token và hiệu quả sử dụng mạng giữa hai cơ chế.

Chuẩn bị tương tự bài 13.1

Gợi ý thực hiện

Bước 1 – Xây dựng CoAP Server hỗ trợ Observe

```
const coap = require('coap');
const server = coap.createServer();
let temperature = 30.0;
let observers = [];
server.on('request', (req, res) => {
  if (req.url === '/sensor/temp') {

    // Nếu client đăng ký Observe
    if (req.headers.Observe === 0) {
      console.log('Client registered Observe');
      observers.push(res);
```

```
      res.write(`temp=${temperature}`);
    }
    else {
      // GET thông thường
      res.end(`temp=${temperature}`);
    }
  }
});

// Giả lập nhiệt độ thay đổi mỗi 5s
setInterval(() => {
  temperature += 0.5;
  console.log('Temperature updated:', temperature);

  observers.forEach(res => {
    res.write(`temp=${temperature}`);
  });
}, 5000);

server.listen(() => {
  console.log('CoAP server started on port 5683');
```

```
});
```

Bước 2 – Xây dựng CoAP Client

- Trường hợp 1 – GET định kỳ (Pull)

```
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>
#include <coap-simple.h>

WiFiUDP udp;
Coap coap(udp);

void callback(CoapPacket &packet, IPAddress ip, int port) {
  Serial.print("Response: ");
  Serial.println((char*)packet.payload);
}

void setup() {
  Serial.begin(115200);
  WiFi.begin("SSID", "PASSWORD");
  while (WiFi.status() != WL_CONNECTED) delay(500);

  coap.response(callback);
  coap.start();
```

```
}


void loop() {
  coap.get(
    IPAddress(192,168,1,100),  // IP Node.js server
    5683,
    "sensor/temp"
  );
  delay(2000);
}
```

- Trường hợp 2 – Observer (Push)

```
void callback(CoapPacket &packet, IPAddress ip, int port) {
  Serial.print("Notify: ");
  Serial.println((char*)packet.payload);
}
void setup() {
  Serial.begin(115200);
  WiFi.begin("SSID", "PASSWORD");
  while (WiFi.status() != WL_CONNECTED) delay(500);
```

```
  coap.response(callback);

  coap.start();



  // Đăng ký Observe

  coap.observe(

    IPAddress(192,168,1,100),

    5683,

    "sensor/temp"

  );

}



void loop() {

  coap.loop();

}
```

Bước 3 – Phân tích bằng Wireshark

- Lọc: coap

Câu hỏi kiểm tra

- Sau khi Observe được thiết lập, ai là bên chủ động gửi dữ liệu? Wireshark thể hiện điều này như thế nào?

- Quan sát các gói Observe:
    - Token có thay đổi không?
    - Message ID có thay đổi không? Giải thích vai trò của từng trường.
- So sánh lưu lượng trong 1 phút:
    - Polling gửi bao nhiêu gói?
    - Observer gửi bao nhiêu gói?
    - Cơ chế nào tiết kiệm băng thông hơn? Vì sao?

Bài 13.3. Thực hành CoAP Blockwise Transfer: Phân tích cơ chế truyền dữ liệu lớn trên nền UDP

Mục đích

Trong bài thực hành này, sinh viên sẽ nghiên cứu cách CoAP truyền dữ liệu lớn vượt quá kích thước MTU bằng cơ chế Blockwise Transfer. Một CoAP Server chạy trên Node.js sẽ cung cấp tài nguyên có payload lớn (ví dụ: dữ liệu cấu hình hoặc firmware giả lập), trong khi CoAP Client trên ESP8266/ESP32 sẽ yêu cầu và nhận dữ liệu theo từng block. Sinh viên sẽ sử dụng Wireshark để quan sát Block1 / Block2 options, phân tích cách CoAP chia nhỏ, đánh số và ghép dữ liệu.

Chuẩn bị tương tự bài 13.1 và 13.2

Gợi ý thực hiện

Bước 1 – Xây dựng CoAP Server với payload lớn

```
const coap = require('coap');
const server = coap.createServer();


const firmware = Buffer.alloc(2048, 'A'); // giả lập firmware 2KB
```

```
server.on('request', (req, res) => {
  if (req.url === '/firmware') {
    console.log('Firmware requested');
    res.end(firmware);
  }
});

server.listen(() => {
  console.log('CoAP Blockwise server started on port 5683');
});
```

Bước 2 – CoAP Client yêu cầu Blockwise

```
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>
#include <coap-simple.h>

WiFiUDP udp;
Coap coap(udp);

void callback(CoapPacket &packet, IPAddress ip, int port) {
  Serial.print("Received block, size=");
  Serial.println(packet.payloadlen);
}
```

```
void setup() {
  Serial.begin(115200);
  WiFi.begin("SSID", "PASSWORD");
  while (WiFi.status() != WL_CONNECTED) delay(500);

  coap.response(callback);
  coap.start();

  // Yêu cầu firmware
  coap.get(
    IPAddress(192,168,1,100), // IP Node.js server
    5683,
    "firmware"
  );
}

void loop() {
  coap.loop();
}
```

Bước 4 – Phân tích Blockwwise bằng Wireshark

- Lọc: **coap**

- Sinh viên quan sát:
    - Option: Block 2
    - Các trường trong Block 2: NUM, M, SZX
    - Nhiều gói Response cho cùng 1 resouce

Bước 5 – Tạo tình huống

- Tắt Wifi client giữa chừng
- Quan sát:
    - Client yêu cầu lại block nào?
    - Block number có thay đổi không?

Câu hỏi kiểm tra

- Vì sao CoAP không sử dụng IP fragmentation mà phải dùng Blockwise Transfer?
- Quan sát Wireshark:
    - Block2 option xuất hiện ở đâu?
    - Ý nghĩa của các trường NUM, M, SZX?
- Nếu một block bị mất:
    - CoAP xử lý thế nào?
    - Có phải gửi lại toàn bộ payload không?
- So sánh Blockwise Transfer và IP fragmentation về:
    - Độ tin cậy
    - Khả năng debug
    - Phù hợp IoT
- Trong bài toán Firmware Update OTA cho IoT:
    - Vì sao Blockwise là bắt buộc?

- Cần kết hợp thêm cơ chế nào để đảm bảo an toàn?

Sinh viên hoàn thành theo gợi ý và trả lời các câu hỏi trên.

HẾT